

Proyecto I: Reconocimiento de comandos de voz y despliegue móvil con ONNX Runtime Mobile

Curso de Inteligencia Artificial
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica

I. OBJETIVO

Diseñar e implementar una solución completa de inteligencia artificial para reconocimiento de comandos de voz, abarcando las etapas de:

- selección de comandos;
- preprocesamiento de audio;
- entrenamiento y comparación de modelos;
- selección del mejor modelo con base en una métrica cuantitativa;
- exportación del modelo a formato ONNX;
- integración del modelo en una aplicación móvil mediante ONNX Runtime Mobile.

En este proyecto deberá construir un sistema de reconocimiento de comandos de voz utilizando el dataset Speech Commands Dataset v0.02. El objetivo final será entrenar, evaluar y seleccionar un modelo de clasificación de audio capaz de reconocer comandos de voz y posteriormente desplegarlo dentro de una **aplicación Android o iOS** utilizando **ONNX Runtime Mobile** para realizar inferencia **local en el dispositivo**.

La aplicación deberá diseñarse como una experiencia de **pantallas navegables**, donde los comandos de voz permitan desplazarse entre opciones, confirmar acciones, controlar dispositivos y activar o detener procesos (Smart Home). De esta forma, la aplicación deberá estar pensada desde la interacción por voz. Adjunto a este proyecto encontrará un prototipo web (html) del diseño de la aplicación y el producto mínimo esperable.

A. Command Selection

Utilizaremos **10 comandos** que son comúnmente utilizados en interfaces por voz. Estos comandos deberán ser el núcleo del sistema de inteligencia artificial empleado en el reconocimiento de voz y deberán estar integrados de manera coherente en el diseño de la aplicación móvil. (ver Tabla I).

TABLE I
LISTA DE COMANDOS AUTORIZADOS

Command	Caso de uso
yes / no	Confirmación
up / down / left / right	Navegación
on / off	Control de dispositivo
stop / go	Control de acción

Concepto de aplicación móvil

La aplicación deberá construirse como una interfaz de navegación por voz inspirada en un escenario realista de interacción *hands-free*. Una propuesta sugerida es una aplicación tipo **Smart Home Voice Navigator**, donde el usuario pueda desplazarse entre pantallas y controlar elementos simulados mediante comandos de voz.

Por ejemplo, la aplicación podría incluir pantallas como:

- **Pantalla principal:** muestra tarjetas o secciones como luces, ventilador, televisión, rutinas o configuración. Ver Figura 3
- **Pantalla de control de dispositivo:** permite encender o apagar un dispositivo usando los comandos *on* y *off*. Ver Figura 4
- **Pantalla de rutinas:** permite iniciar o detener acciones con *go* y *stop*. Ver Figura 5
- **Pantalla de confirmación:** utiliza *yes* y *no* para aceptar o cancelar acciones sensibles. Ver Figura 6

En este diseño, los comandos *up*, *down*, *left* y *right* deberán utilizarse para mover el foco visual entre opciones, mientras que *yes* y *no* deberán emplearse para confirmar o cancelar acciones. Los comandos *on* y *off* deberán actuar sobre dispositivos o estados, y *go* y *stop* deberán utilizarse para iniciar o detener acciones o rutinas.

Cada equipo deberá construir un sistema de clasificación de audio que reconozca estos 10 comandos y los integre de manera funcional dentro de una aplicación móvil navegable.

Importante:

- La aplicación no debe depender de servicios externos, APIs en la nube ni procesamiento remoto.
- La inferencia debe ejecutarse **localmente en el dispositivo** utilizando **ONNX Runtime Mobile**.
- La aplicación debe reflejar un diseño coherente con el uso de los comandos seleccionados.
- No se espera únicamente una prueba técnica del modelo, sino una experiencia funcional de navegación e interacción basada en voz.

II. DESCRIPCION DEL DATASET

Para el desarrollo de este proyecto se utilizará un conjunto de datos público de clasificación de audio. Este dataset contiene grabaciones de corta duración con diferentes categorías sonoras. Cada clip de audio se encuentra etiquetado con una

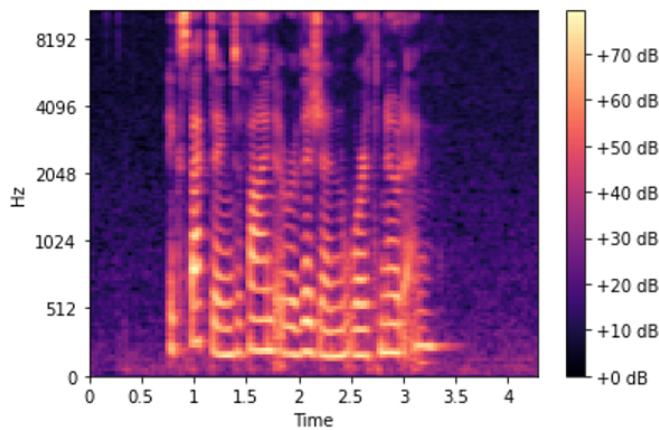


Fig. 1. Espectrograma de audio a modo de ejemplo

clase, lo cual permite entrenar modelos supervisados para reconocer patrones acústicos, por lo tanto su objetivo es desarrollar modelos que permitan reconocer estos patrones para crear un modelo de clasificación multiclase.

Previo al entrenamiento, las grabaciones deben ser procesadas para obtener una representación en formato de imagen como se observa en la Figura 1. Estas representaciones servirán como entrada para los modelos convolucionales implementados ¹

III. DISEÑO DE ARQUITECTURA DE IA

Se deben construir manualmente dos modelos con **PyTorch**, sin utilizar librerías de alto nivel que abstraigan la definición de capas más allá de `torch.nn`. Es decir deben ser creadas por su equipo de trabajo.

Cada modelo debe incluir: definición de arquitectura (y un respectivo diagrama), función de pérdida, optimizador y rutinas de entrenamiento/validación y test. Además, se exige el control de aleatoriedad, registro de métricas y código reproducible.

Modelo A: LeNet-5 clásico

El **Modelo A** debe ser una variante clásica de *LeNet-5*, adaptada a clasificación de audio a partir de representaciones espectrales.

Se permite alterar el tamaño de entrada, el número de canales, y los hiperparámetros. Se deberá consultar al profesor cualquier otra modificación de la arquitectura original, y además debe de ser debidamente justificada en el informe.

Modelo B: Arquitectura alternativa basada en literatura

El **Modelo B** puede implementar cualquier arquitectura *distinta* basada en la literatura o un diseño propio **fundamentado académicamente**.

Entre las referencias sugeridas se encuentran *EfficientNet*, *MobileNet*, *Inception*, *DenseNet* o *ResNet*.

¹Enlace para descarga del dataset se encuentra disponible aquí

En caso de proponer un modelo propio, deberá explicarse claramente la motivación, las decisiones de diseño (profundidad, anchos de bloque, tipo de bloques, activaciones, etc) y cómo estas afectan la capacidad de generalización y la eficiencia computacional.

Recuerde que la selección del modelo debe ser en función del caso de uso de una aplicación móvil.

A. Preprocesamiento de los datos

Los datos son un conjunto de audios los cuales deben de ser transformados para poder utilizarse dentro de las redes neuronales (CNN) propuestas, puestas reciben como entrada imágenes y los datos actuales son grabaciones de audio.

Para el entrenamiento debe de utilizar dos conjuntos de datos claramente diferenciados para mejorar la experimentación y rendimiento de los modelos propuestos.

- El primer conjunto son los datos crudos luego de convertir los formatos de audio a imágenes (crudo).
- El segundo conjunto es igual que el primero, con la excepción de que debe de aplicar técnicas de data augmentation inspirada en audio [1]. **Debidamente justificada con literatura** (aumentado).

IV. ENTRENAMIENTO

Debe entrenar 4 distintos modelos base Modelo A/Base, Modelo A/Aumentado, Modelo B/Base, Modelo/Aumentado.

Para cada uno de ellos se deben de realizar al menos 3 entrenamientos con diferentes hiperparámetros para seleccionar el mejor de cada combinación. Muestre los resultados de los entrenamientos y set de validación para luego hacer comparación de resultados de los mejores modelos de cada uno y luego compare los mejores modelos base. Se espera que se muestren los resultados de los 12 modelos entrenados.

Recuerde que para cada dataset debe dividirlo en 3 partes (entrenamiento, validación y testing), durante el entrenamiento de su modelo debe verificar que no tenga problemas de overfitting o underfitting sus modelos, al igual que justificarlos por escrito en la investigación.

Recomendaciones: En caso de no contar con GPU en una unidad local, pueden utilizar Google Colab para realizar el proyecto, en este caso debe subir el dataset dentro de Google Drive para poder accederlo guía.

V. EVALUACIÓN DE MODELOS

Debe comparar los resultados de los modelos para indicar cuál es el mejor, para realizar esta comparación debe utilizar la herramienta de Weights and Biases, esta herramienta tracea y visualiza varios aspectos del proceso de entrenamiento del modelo en tiempo real como el overfitting, loss, accuracy, F1-Score, matriz de confusión y otras métricas de evaluación como se observa en la Figura 2.

TABLE II
RÚBRICA DE PROYECTO

Criterio	Puntaje Máx.
Diseño y arquitectura definida Modelo A	5
Diseño y arquitectura definida Modelo B	10
Selección de técnicas de aumentación	10
Entrenamiento con datos crudos y aumentados Modelo A	15
Entrenamiento con datos crudos y aumentados Modelo B	15
Presentación de métricas y selección del mejor modelo A (test)	10
Presentación de métricas y selección del mejor modelo B (test)	10
Comparación de modelos finales	10
Integración en aplicación móvil	15
Total	100

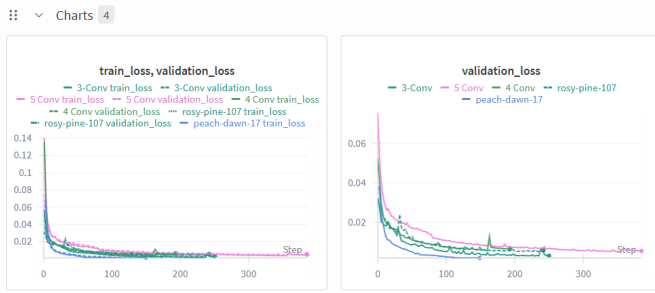


Fig. 2. Imagen de la plataforma interactiva de WandB

VI. EXPORTACIÓN DE MODELO A MÓVIL

Una vez identificado el mejor modelo con base a las métricas seleccionadas, el siguiente paso consiste en prepararlo para despliegue móvil. Para ello, el estudiante deberá congelar la versión final del modelo, conservar sus pesos entrenados y documentar claramente la configuración utilizada. Además, será necesario definir de forma precisa el pipeline de entrada que usará el sistema en producción, incluyendo formato del audio, frecuencia de muestreo, preprocesamiento y forma esperada del tensor de entrada.

Posteriormente, el modelo deberá colocarse en modo de inferencia y exportarse al formato **ONNX**, verificando que las entradas y salidas del archivo generado coincidan con las del modelo original. No basta con generar el archivo `.onnx`; también será necesario comprobar que puede cargarse correctamente y que produce predicciones consistentes con las obtenidas en el entorno de entrenamiento. Esta validación puede realizarse comparando resultados del modelo original y del modelo exportado sobre ejemplos de prueba.

Finalmente, una vez validado el archivo ONNX, este deberá integrarse en la aplicación mediante **ONNX Runtime Mobile**. En esta etapa, el estudiante deberá asegurarse de que la aplicación reproduce correctamente el mismo preprocesamiento utilizado durante el entrenamiento y que la salida del modelo se transforma de manera coherente en acciones dentro de la

interfaz.

VII. ENTREGA

El informe deberá realizarse en $\text{\LaTeX}$ (Overleaf) utilizando la plantilla IEEE para artículos científicos. El documento debe contener visualizaciones, y la interpretación de los resultados. Además, se debe adjuntar un **Jupyter Notebook** con el código implementado para el desarrollo del modelo de inteligencia artificial. Código de su aplicación móvil integrado con los comandos de AI. La entrega final consistirá en un archivo comprimido (`.zip`) que contenga:

- Código fuente en $\text{\LaTeX}$.
- El PDF del informe.
- El Notebook con el código fuente.
- Código de la aplicación móvil.

RÚBRICA

Si el trabajo no se encuentra debidamente ordenado y presentado siguiendo una adecuada estructura para el informe, puede ser considerado como incompleto y cualquiera de las rúbricas se puede ver afectada ver Tabla II. Esto quiere decir que se evaluará el contenido del informe y se verificará contra los notebooks, y el desempeño de la aplicación.

ANEXOS

REFERENCES

- [1] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," *arXiv preprint arXiv:1904.08779*, 2019.

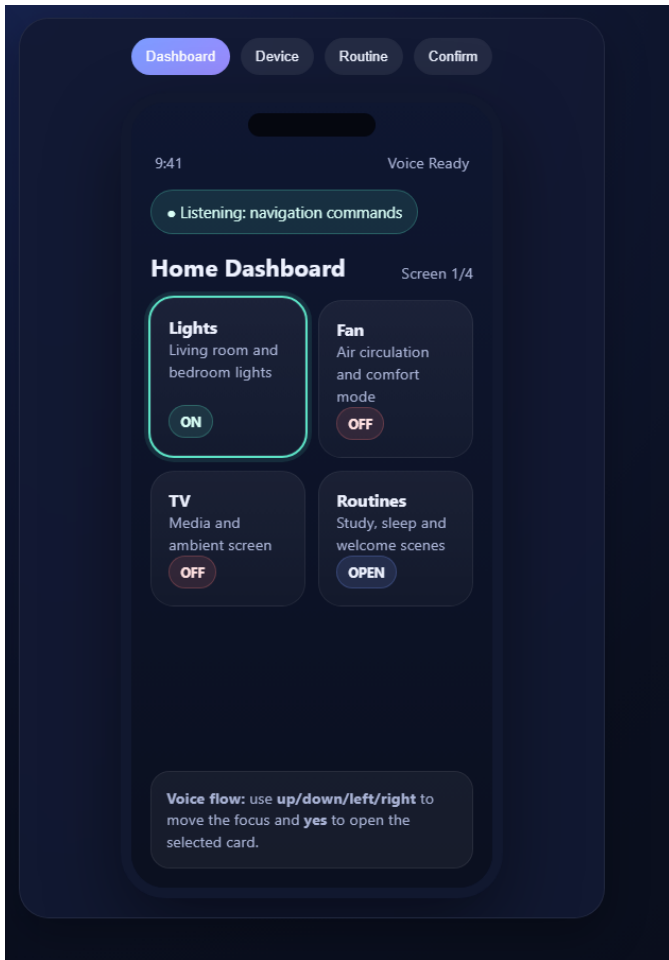


Fig. 3. Dashboard Screen

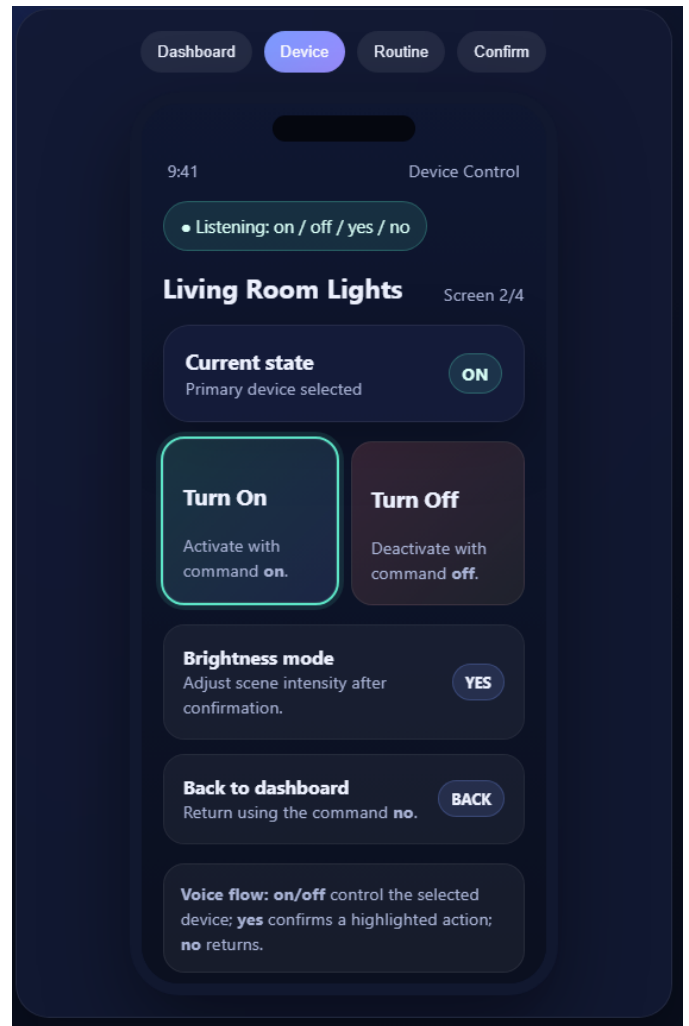


Fig. 4. Device Screen

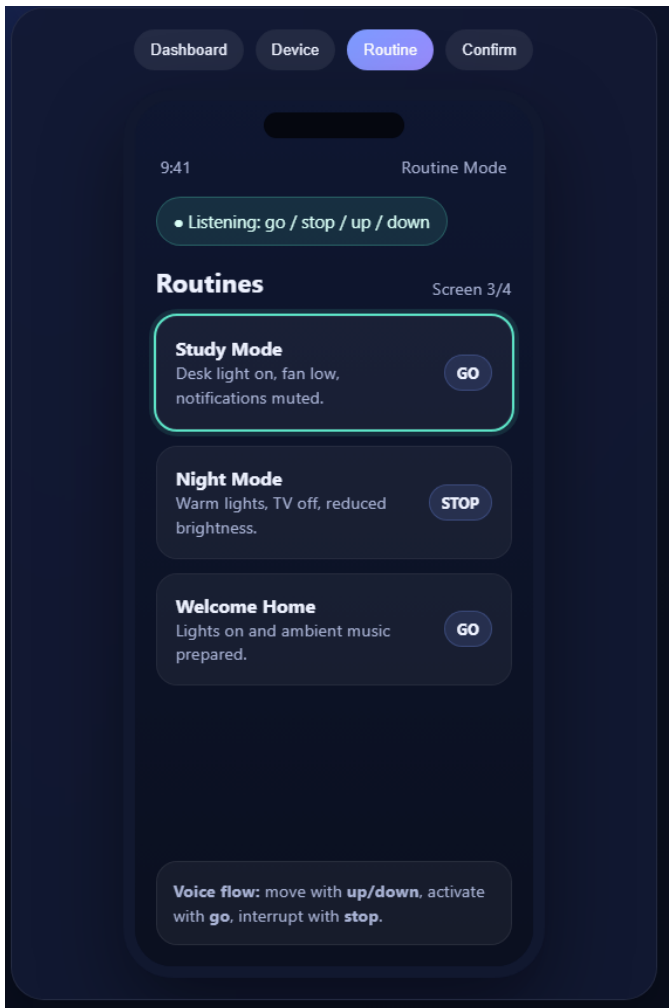


Fig. 5. Routine Screen

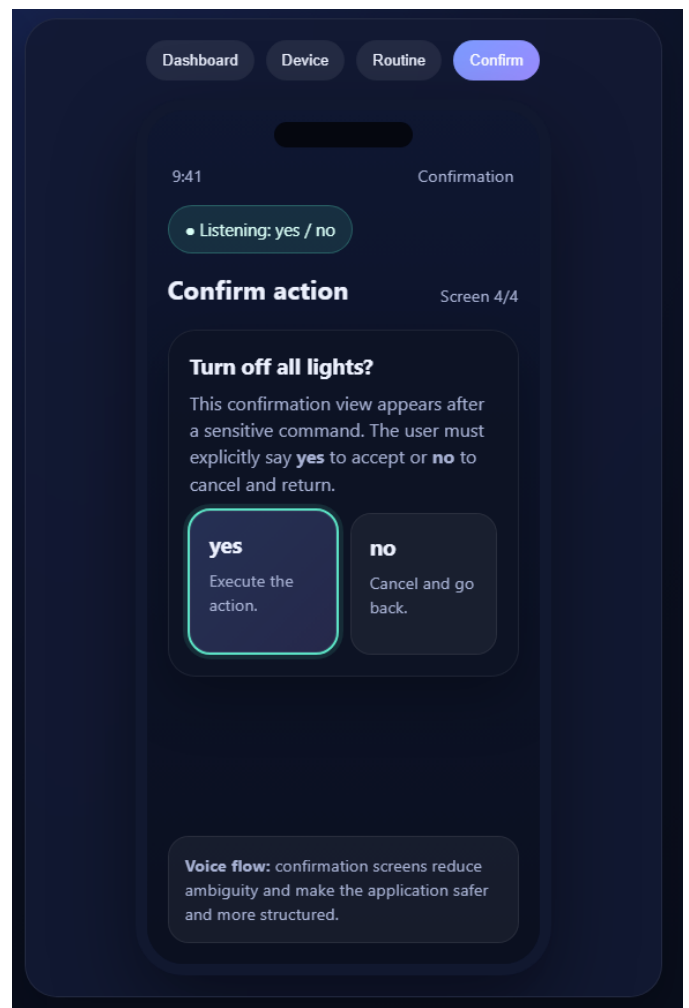


Fig. 6. Confirm Screen